

```

Task: Mine 3 wood logs

RESPONSE:
{
  "reasoning": "You need to mine 3 wood logs. You have 2 oak logs
and 2 spruce logs, which add up to 4 wood logs.",
  "success": true,
  "critique": ""
}

INPUT:
Inventory (3/36): {'crafting_table': 1, 'spruce_planks': 6, 'stick':
4}

Task: Craft a wooden pickaxe

RESPONSE:
{
  "reasoning": "You have enough materials to craft a wooden pickaxe,
but you didn't craft it.",
  "success": false,
  "critique": "Craft a wooden pickaxe with a crafting table using 3
spruce planks and 2 sticks."
}

INPUT:
Inventory (2/36): {'raw_iron': 5, 'stone_pickaxe': 1}

Task: Mine 5 iron_ore

RESPONSE:
{
  "reasoning": "Mining iron_ore in Minecraft will get raw_iron. You
have 5 raw_iron in your inventory.",
  "success": true,
  "critique": ""
}

INPUT:
Biome: plains

Nearby blocks: stone, dirt, grass_block, grass, farmland, wheat

Inventory (26/36): ...

Task: Plant 1 wheat seed.

RESPONSE:
{
  "reasoning": "For planting tasks, inventory information is useless
. In nearby blocks, there is farmland and wheat, which means you
succeed to plant the wheat seed.",
  "success": true,
  "critique": ""
}

INPUT:
Inventory (11/36): {... , 'rotten_flesh': 1}

Task: Kill 1 zombie

Context: ...

RESPONSE
{

```

```

    "reasoning": "You have rotten flesh in your inventory, which means
    you successfully killed one zombie.",
    "success": true,
    "critique": ""
}

INPUT:
Hunger: 20.0/20.0

Inventory (11/36): ...

Task: Eat 1 ...

Context: ...

RESPONSE
{
    "reasoning": "For all eating task, if the player's hunger is 20.0,
    then the player successfully ate the food.",
    "success": true,
    "critique": ""
}

INPUT:
Nearby blocks: chest

Inventory (28/36): {'rail': 1, 'coal': 2, 'oak_planks': 13, '
    copper_block': 1, 'diorite': 7, 'cooked_beef': 4, 'granite': 22, '
    cobbled_deepslate': 23, 'feather': 4, 'leather': 2, '
    cooked_chicken': 3, 'white_wool': 2, 'stick': 3, 'black_wool': 1,
    'stone_sword': 2, 'stone_hoe': 1, 'stone_axe': 2, 'stone_shovel':
    2, 'cooked_mutton': 4, 'cobblestone_wall': 18, 'crafting_table':
    1, 'furnace': 1, 'iron_pickaxe': 1, 'stone_pickaxe': 1, '
    raw_copper': 12}

Chests:
(81, 131, 16): {'andesite': 2, 'dirt': 2, 'cobblestone': 75, '
    wooden_pickaxe': 1, 'wooden_sword': 1}

Task: Deposit useless items into the chest at (81, 131, 16)

Context: ...

RESPONSE
{
    "reasoning": "You have 28 items in your inventory after depositing
    , which is more than 20. You need to deposit more items from your
    inventory to the chest.",
    "success": false,
    "critique": "Deposit more useless items such as copper_block,
    diorite, granite, cobbled_deepslate, feather, and leather to meet
    the requirement of having only 20 occupied slots in your inventory
    ."
}

```

A.6 System-level Comparison between VOYAGER and Prior Works

We make a system-level comparison in Table. A.2. Voyager stands out as the only method featuring a combination of automatic curriculum, iterative planning, and a skill library. Moreover, it learns to play Minecraft without the need for any gradient update.

Table A.2: System-level comparison between VOYAGER and prior works.

	VPT [8]	DreamerV3 [69]	DECKARD [53]	DEPS [55]	Plan4MC [71]	VOYAGER
Demos	Videos	None	Videos	None	None	None
Rewards	Sparse	Dense	Sparse	None	Dense	None
Observations	Pixels Only	Pixels & Meta	Pixels & Inventory	Feedback & Inventory	Pixels & Meta	Feedback & Meta & Inventory
Actions	Keyboard & Mouse	Discrete	Keyboard & Mouse	Keyboard & Mouse	Discrete	Code
Automatic Curriculum			✓			✓ (in-context GPT-4 proposal)
Iterative Planning				✓		✓ (3 types of feedback)
Skill Library					✓ (pre-defined)	✓ (self-generated)
Gradient-Free						✓

B Experiments

B.1 Experimental Setup

Our simulation environment is built upon MineDojo [23] and utilizes Mineflayer [52] JavaScript APIs for motor controls (Sec. A.4.2). Additionally, we incorporate many `bot.chat()` into Mineflayer functions to provide abundant environment feedback and implement various condition checks along with try-catch exceptions for continuous execution. If the bot dies, it is resurrected near the closest ground, and its inventory is preserved for uninterrupted exploration. The bot recycles its crafting table and furnace after program execution. For detailed implementations, please refer to our codebase.

B.2 Baselines

ReAct [29] uses chain-of-thought prompting [46] by generating both reasoning traces and action plans with LLMs. We provide it with our environment feedback and the agent states as observations. ReAct undergoes one round of code generation from scratch, followed by three rounds of code refinement. This process is then repeated until the maximum prompting iteration is reached.

Reflexion [30] is built on top of ReAct [29] with self-reflection to infer more intuitive future actions. We provide it with environment feedback, the agent states, execution errors, and our self-verification module. Similar to ReAct, Reflexion undergoes one round of code generation from scratch, followed by three rounds of code refinement. This process is then repeated until the maximum prompting iteration is reached.

AutoGPT [28] is a popular software tool that automates NLP tasks by decomposing a high-level goal into multiple subgoals and executing them in a ReAct-style loop. We re-implement AutoGPT by using GPT-4 to do task decomposition and provide it with the agent states, environment feedback, and execution errors as observations for subgoal execution. Compared with VOYAGER, AutoGPT lacks the skill library for accumulating knowledge, self-verification for assessing task success, and automatic curriculum for open-ended exploration. During each subgoal execution, if no execution error occurs, we consider the subgoal completed and proceed to the next one. Otherwise, we refine the program until three rounds of code refinement (equivalent to four rounds of code generation) are completed, and then move on to the next subgoal. If three consecutive subgoals do not result in acquiring a new item, we replan by rerunning the task decomposition.

The task is “explore the world and get as many items as possible” for all baselines.

Table A.3: Comparison between VOYAGER and baselines.

	ReAct [29]	Reflexion [30]	AutoGPT [28]	VOYAGER
Chain-of-Thought [46]	✓	✓	✓	✓
Self Verification		✓		✓
Environment Feedback	✓	✓	✓	✓
Execution Errors		✓	✓	✓
Agent State	✓	✓	✓	✓
Skill Library				✓
Automatic Curriculum				✓

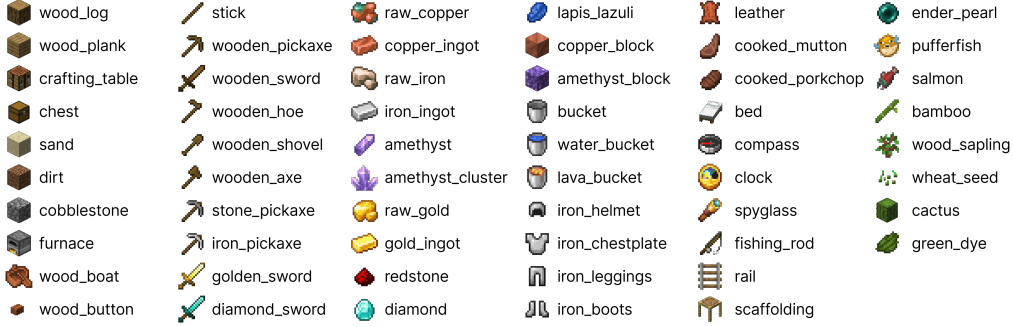


Figure A.1: Minecraft item icons with corresponding names.

B.3 Ablations

We ablate 6 design choices (automatic curriculum, skill library, environment feedback, execution errors, self-verification, and GPT-4 for code generation) in VOYAGER and study their impact on exploration performance.

- **Manual Curriculum:** We substitute the automatic curriculum with a manually designed curriculum for mining a diamond: “Mine 3 wood log”, “Craft 1 crafting table”, “Craft 1 wooden pickaxe”, “Mine 11 cobblestone”, “Craft 1 stone pickaxe”, “Craft 1 furnace”, “Mine 3 iron ore”, “Smelt 3 iron ore”, “Craft 1 iron pickaxe”, “Mine 1 diamond”. A manual curriculum requires human effort to design and is not scalable for open-ended exploration.
- **Random Curriculum:** We curate 101 items obtained by VOYAGER and create a random curriculum by randomly selecting one item as the next task.
- **w/o Skill Library:** We remove the skill library, eliminating skill retrieval for code generation.
- **w/o Environment Feedback:** We exclude environment feedback (chat log) from the prompt for code generation.
- **w/o Execution Errors:** We exclude execution errors from the prompt for code generation.
- **w/o Self-Verification:** For each task, we generate code without self-verification and iteratively refine the program for 3 rounds (equivalent to 4 rounds of code generation in total).
- **GPT-3.5:** We replace GPT-4 with GPT-3.5 for code generation. We retain GPT-4 for the automatic curriculum and the self-verification module.

B.4 Evaluation Results

B.4.1 Significantly Better Exploration

The meaning of each icon in Fig. 1 is shown in Fig. A.1.

We run three trials for each method. The items collected by VOYAGER in each trial is

- **Trial 1:** 'iron_ingot', 'stone_shovel', 'iron_leggings', 'fishing_rod', 'pufferfish', 'oak_log', 'cooked_mutton', 'green_dye', 'flint', 'chest', 'iron_sword', 'string', 'ender_pearl', 'raw_copper', 'crafting_table', 'cactus', 'lapis_lazuli', 'iron_pickaxe', 'copper_ingot', 'stone_pickaxe', 'wooden_hoe', 'scaffolding', 'stick', 'porkchop', 'copper_block', 'gravel', 'grass_block', 'white_bed', 'bone', 'dirt', 'mutton', 'white_wool', 'oak_sapling', 'coal', 'bamboo', 'wooden_pickaxe', 'rotten_flesh', 'cooked_porkchop', 'cod', 'iron_boots', 'lightning_rod', 'diorite', 'water_bucket', 'shears', 'furnace', 'andesite', 'granite', 'bucket', 'wooden_sword', 'sandstone', 'iron_helmet', 'raw_iron', 'sand', 'acacia_log', 'cooked_cod', 'oak_planks', 'azure_bluet', 'iron_shovel', 'acacia_planks', 'shield', 'iron_axe', 'iron_chestplate', 'cobblestone';
- **Trial 2:** 'iron_ingot', 'tuff', 'stone_shovel', 'iron_leggings', 'fishing_rod', 'cooked_mutton', 'spruce_planks', 'gunpowder', 'amethyst_shard', 'chest', 'string', 'cooked_salmon', 'iron_sword', 'raw_copper', 'crafting_table', 'torch', 'lapis_lazuli', 'iron_pickaxe', 'copper_ingot', 'stone_pickaxe', 'wooden_hoe', 'stick', 'amethyst_block', 'salmon', 'calcite', 'gravel', 'white_bed', 'bone', 'dirt', 'mutton', 'white_wool', 'spyglass', 'coal', 'wooden_pickaxe', 'cod', 'iron_boots', 'lily_pad', 'cobbledeepslate', 'lightning_rod', 'snowball', 'stone_axe', 'smooth_basalt', 'diorite', 'water_bucket', 'furnace', 'andesite', 'bucket', 'granite', 'shield', 'iron_helmet', 'raw_iron', 'cobblestone', 'spruce_log', 'cooked_cod', 'tripwire_hook', 'stone_hoe', 'iron_chestplate', 'stone_sword';
- **Trial 3:** 'spruce_planks', 'dirt', 'shield', 'redstone', 'clock', 'diamond_sword', 'iron_chestplate', 'stone_pickaxe', 'leather', 'string', 'chicken', 'chest', 'diorite', 'iron_leggings', 'black_wool', 'cobblestone_wall', 'cobblestone', 'cooked_chicken', 'feather', 'stone_sword', 'raw_gold', 'gravel', 'birch_planks', 'coal', 'cobbledeepslate', 'oak_planks', 'iron_pickaxe', 'granite', 'tuff', 'crafting_table', 'iron_helmet', 'stone_hoe', 'iron_ingot', 'stone_axe', 'birch_boat', 'stick', 'sand', 'bone', 'raw_iron', 'beef', 'rail', 'oak_sapling', 'kelp', 'gold_ingot', 'birch_log', 'wheat_seeds', 'cooked_mutton', 'furnace', 'arrow', 'stone_shovel', 'white_wool', 'andesite', 'jungle_slab', 'mutton', 'iron_sword', 'copper_ingot', 'diamond', 'torch', 'oak_log', 'cooked_beef', 'copper_block', 'flint', 'bone_meal', 'raw_copper', 'wooden_pickaxe', 'iron_boots', 'wooden_sword'.

The items collected by ReAct [29] in each trial is

- **Trial 1:** 'bamboo', 'dirt', 'sand', 'wheat_seeds';
- **Trial 2:** 'dirt', 'rabbit', 'spruce_log', 'spruce_sapling';
- **Trial 3:** 'dirt', 'pointed_dripstone';

The items collected by Reflexion [30] in each trial is

- **Trial 1:** 'crafting_table', 'orange_tulip', 'oak_planks', 'oak_log', 'dirt';
- **Trial 2:** 'spruce_log', 'dirt', 'clay_ball', 'sand', 'gravel';
- **Trial 3:** 'wheat_seeds', 'oak_log', 'dirt', 'birch_log', 'sand'.

The items collected by AutoGPT [28] in each trial is

- **Trial 1:** 'feather', 'oak_log', 'leather', 'stick', 'porkchop', 'chicken', 'crafting_table', 'wheat_seeds', 'oak_planks', 'dirt', 'mutton';
- **Trial 2:** 'wooden_pickaxe', 'iron_ingot', 'stone', 'coal', 'spruce_planks', 'string', 'raw_copper', 'crafting_table', 'diorite', 'andesite', 'furnace', 'torch', 'spruce_sapling', 'granite', 'iron_pickaxe', 'stone_pickaxe', 'wooden_axe', 'raw_iron', 'stick', 'spruce_log', 'dirt', 'cobblestone';
- **Trial 3:** 'wooden_shovel', 'wooden_pickaxe', 'iron_ingot', 'stone', 'cod', 'coal', 'oak_log', 'flint', 'raw_copper', 'crafting_table', 'diorite', 'furnace', 'andesite', 'torch', 'granite', 'lapis_lazuli', 'iron_pickaxe', 'stone_pickaxe', 'raw_iron', 'stick', 'gravel', 'oak_planks', 'dirt', 'iron_axe', 'cobblestone'.